
بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

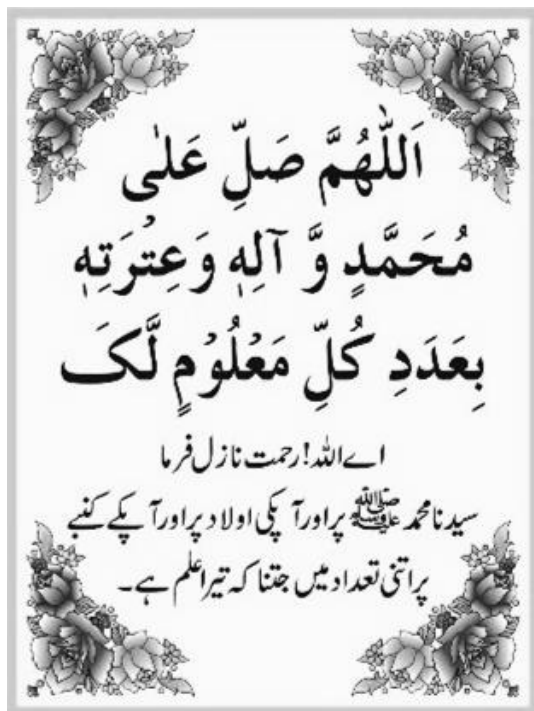


Machine Learning

Lecture 07 – Decision Tree Learning

Dr. Jawad Shafi

Dua – Take Help from Allah Before Starting Any Task



اللَّهُمَّ خِرْ لِي وَاخْتَرْ لِي

سُبْحَانَكَ لَا عِلْمَ لَنَا إِلَّا مَا عَلَّمْتَنَا

إِنَّكَ أَنْتَ الْعَلِيمُ الْحَكِيمُ

رَبِّ اشْرَحْ لِي صَدْرِي

وَيَسِّرْ لِي أَمْرِي

وَاخْلُلْ عُقْدَةً مِنْ لِسَانِي

يَفْقَهُوا قَوْلِي

Decision Tree

- The general motive of using Decision Tree is to create a training model which can use to predict class or value of target variables by **learning decision rules** inferred from prior data (training data).
- Decision Tree algorithm belongs to the family of supervised learning algorithms. Unlike other supervised learning algorithms, decision tree algorithm can be used to solve both regression and classification problems.

Decision Tree Conti.

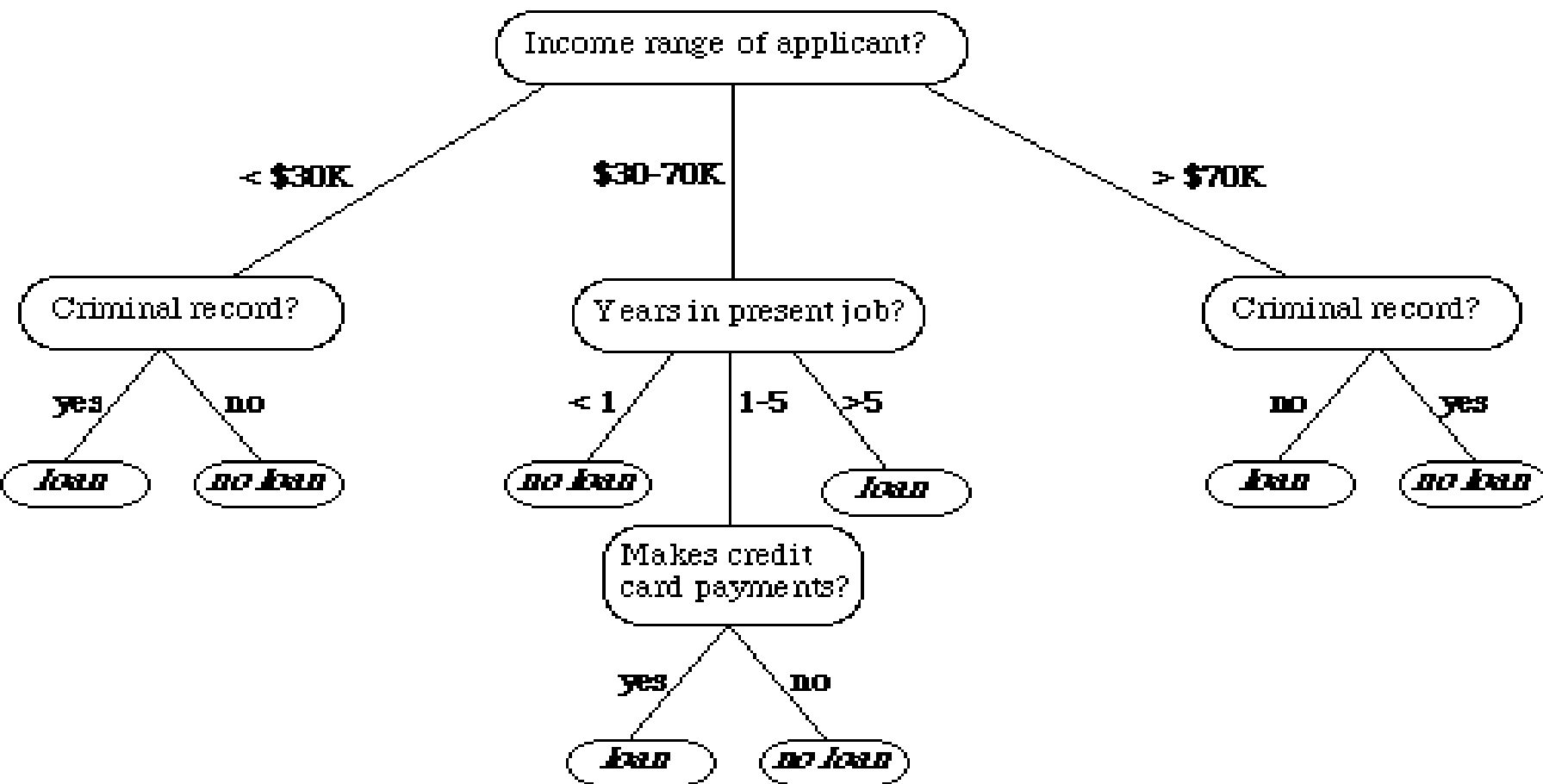
- The understanding level of Decision Trees algorithm is easy as compared with other classification algorithms
 - The decision tree algorithm tries to solve the problem, by using tree representation.
 - Each **internal node** of the tree corresponds to an attribute, and
 - each **leaf node** corresponds to a class label.

Why Decision Tree

- Decision tree often mimic the human level thinking so its so simple to understand the data and make some good interpretations
- Decision tree actually make you see the logic for the data to interpret (not like black box algorithms like SVM, NN, etc..)

Decision Tree: Example

If we are classifying *bank loan* application for a customer, the decision tree may look like as:



DT: Example Conti.

- The whole idea is to create a tree like this for the entire data and to process a single outcome at every leaf (or minimize the error in every leaf)

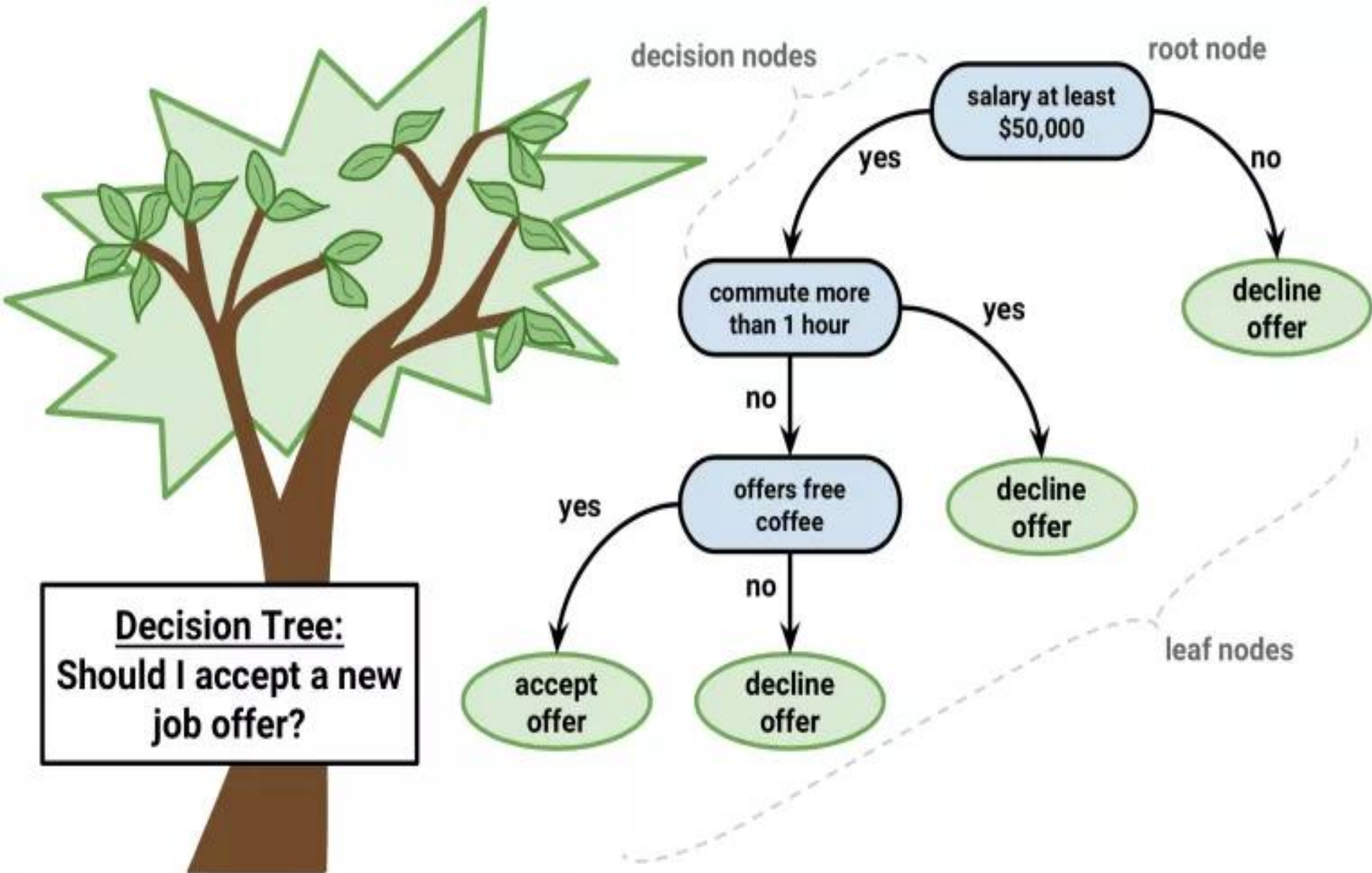
DT: Definition

- A decision tree is a tree where each node represents a feature (attribute), each link (branch) represents a decision (rule) and each leaf represents an outcome (categorical or continuous value)

DT Algorithm: Pseudo code

- 1- Place the best attribute of the dataset at the **root** of the tree.
- 2- Split the training set into **subsets**. Subsets should be made in such a way that each subset contains data with the same value for an attribute.
- 3- Repeat step 1 and step 2 on each subset until you find **leaf nodes** in all the branches of the tree.

Decision Tree Algorithm Pseudo code



Algorithms to build a decision tree

- CART (Classification and Regression Trees) → uses ***Gini Index (Classification)*** as metric.
- ID3 (Iterative Dichotomiser 3) → uses ***Entropy function*** and **Information gain** as metrics.

*Classification using the **ID3** algorithm*

Let's take a famous dataset in the machine learning world which is weather dataset (playing game Y or N based on weather condition)

ID3: Weather Dataset

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

ID3: Conti.

- We have four X values (outlook, temp, humidity and windy) being categorical and one y value (play Y or N) also being categorical.
- This is a binary classification problem, lets build the tree using the **ID3** algorithm
- To create a tree, we need to have a root node first and we know that nodes are features / attributes (outlook, temp, humidity and windy)

Which one do we need to pick first??

Outlook , Temp, Humidity, or Windy

- Determine the attribute that best classifies the training data; use this attribute at the root of the tree. Repeat this process at for each branch.
- This means we are performing top-down, greedy search through the space of possible decision trees.

How to choose the best attribute?

Use the attribute with the highest *information gain* in *ID3*

Entropy

- *To define information gain precisely, we begin by defining a measure commonly used in information theory, called **entropy** that characterizes the (im)purity of an arbitrary collection of examples.*

Entropy

Entropy $H(S)$ is a measure of the amount of uncertainty in the (data) set S (i.e. entropy characterizes the (data) set S).

$$H(S) = \sum_{c \in C} -p(c) \log_2 p(c)$$

Where,

- S – The current (data) set for which entropy is being calculated (changes every iteration of the ID3 algorithm)
- C – Set of classes in S $C = \{ \text{yes, no} \}$
- $p(c)$ – The proportion of the number of elements in class c to the number of elements in set S

When $H(S) = 0$, the set S is perfectly classified (i.e. all elements in S are of the same class).

In ID3, entropy is calculated for each remaining attribute. The attribute with the **smallest** entropy is used to split the set S on this iteration. The higher the entropy, the higher the potential to improve the classification here.

For a Binary Classification Problem

- If all examples are positive or all are negative then entropy will be **zero** i.e, low.
- If half of the examples are of positive class and half are of negative class then entropy is **one** i.e. high.

Information Gain

Information gain

Information gain $IG(A)$ is the measure of the difference in entropy from before to after the set S is split on an attribute A . In other words, how much uncertainty in S was reduced after splitting set S on attribute A .

$$IG(A, S) = H(S) - \sum_{t \in T} p(t)H(t)$$

Where,

- $H(S)$ – Entropy of set S
- T – The subsets created from splitting set S by attribute A such that $S = \bigcup_{t \in T} t$
- $p(t)$ – The proportion of the number of elements in t to the number of elements in set S
- $H(t)$ – Entropy of subset t

In ID3, information gain can be calculated (instead of entropy) for each remaining attribute. The attribute with the **largest** information gain is used to split the set S on this iteration.

- **Lets apply these metrics to our dataset to split the data (getting the root node)**

Steps:

1. Compute the entropy for data-set
2. For every attribute / feature:
 1. Calculate entropy for all categorical values
 2. Take average information entropy for the current attribute
 3. Calculate gain for the current attribute
3. Pick the highest gain attribute.
4. Repeat until we get the tree we desired.

Compute the entropy for the weather data set:

$$H(S) = \sum_{c \in C} -p(c) \log_2 p(c)$$

$$C = \{\text{yes}, \text{no}\}$$

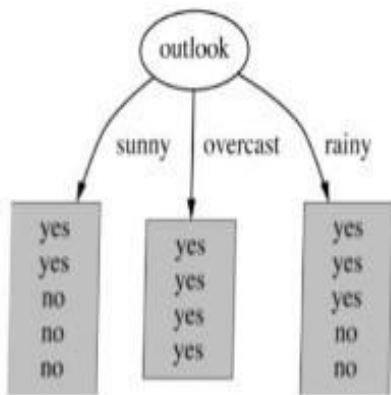
Out of 14 instances, 9 are classified as yes, and 5 as no

$$p_{\text{yes}} = -(9/14) * \log_2(9/14) = 0.41$$

$$p_{\text{no}} = -(5/14) * \log_2(5/14) = 0.53$$

$$H(S) = p_{\text{yes}} + p_{\text{no}} = 0.94$$

For every feature calculate the entropy and information gain



$$E(\text{Outlook}=\text{sunny}) = -\frac{2}{5} \log\left(\frac{2}{5}\right) - \frac{3}{5} \log\left(\frac{3}{5}\right) = 0.971$$

$$E(\text{Outlook}=\text{overcast}) = -1 \log(1) - 0 \log(0) = 0$$

$$E(\text{Outlook}=\text{rainy}) = -\frac{3}{5} \log\left(\frac{3}{5}\right) - \frac{2}{5} \log\left(\frac{2}{5}\right) = 0.971$$

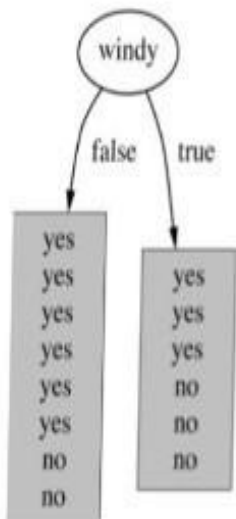
$$\left. \begin{array}{l} E(\text{Outlook}=\text{sunny}) \\ E(\text{Outlook}=\text{overcast}) \\ E(\text{Outlook}=\text{rainy}) \end{array} \right\} H(S, \text{Outlook})$$

Average Entropy information for Outlook

$$I(\text{Outlook}) = \frac{5}{14} * 0.971 + \frac{4}{14} * 0 + \frac{5}{14} * 0.971 = 0.693$$

$$\text{Gain}(\text{Outlook}) = E(S) - I(\text{outlook}) = 0.94 - 0.693 = 0.247$$

$$\left. \begin{array}{l} \sum_{t \in T} p(t) H(t) \\ \Rightarrow IG(A, S) = H(S) - \sum_{t \in T} p(t) H(t) \end{array} \right\}$$



$$E(\text{Windy}=\text{false}) = -\frac{6}{8} \log\left(\frac{6}{8}\right) - \frac{2}{8} \log\left(\frac{2}{8}\right) = 0.811$$

$$E(\text{Windy}=\text{true}) = -\frac{3}{6} \log\left(\frac{3}{6}\right) - \frac{3}{6} \log\left(\frac{3}{6}\right) = 1$$

Average entropy information for Windy

$$I(\text{Windy}) = \frac{8}{14} * 0.811 + \frac{6}{14} * 1 = 0.892$$

$$\text{Gain}(\text{Windy}) = E(S) - I(\text{Windy}) = 0.94 - 0.892 = 0.048$$

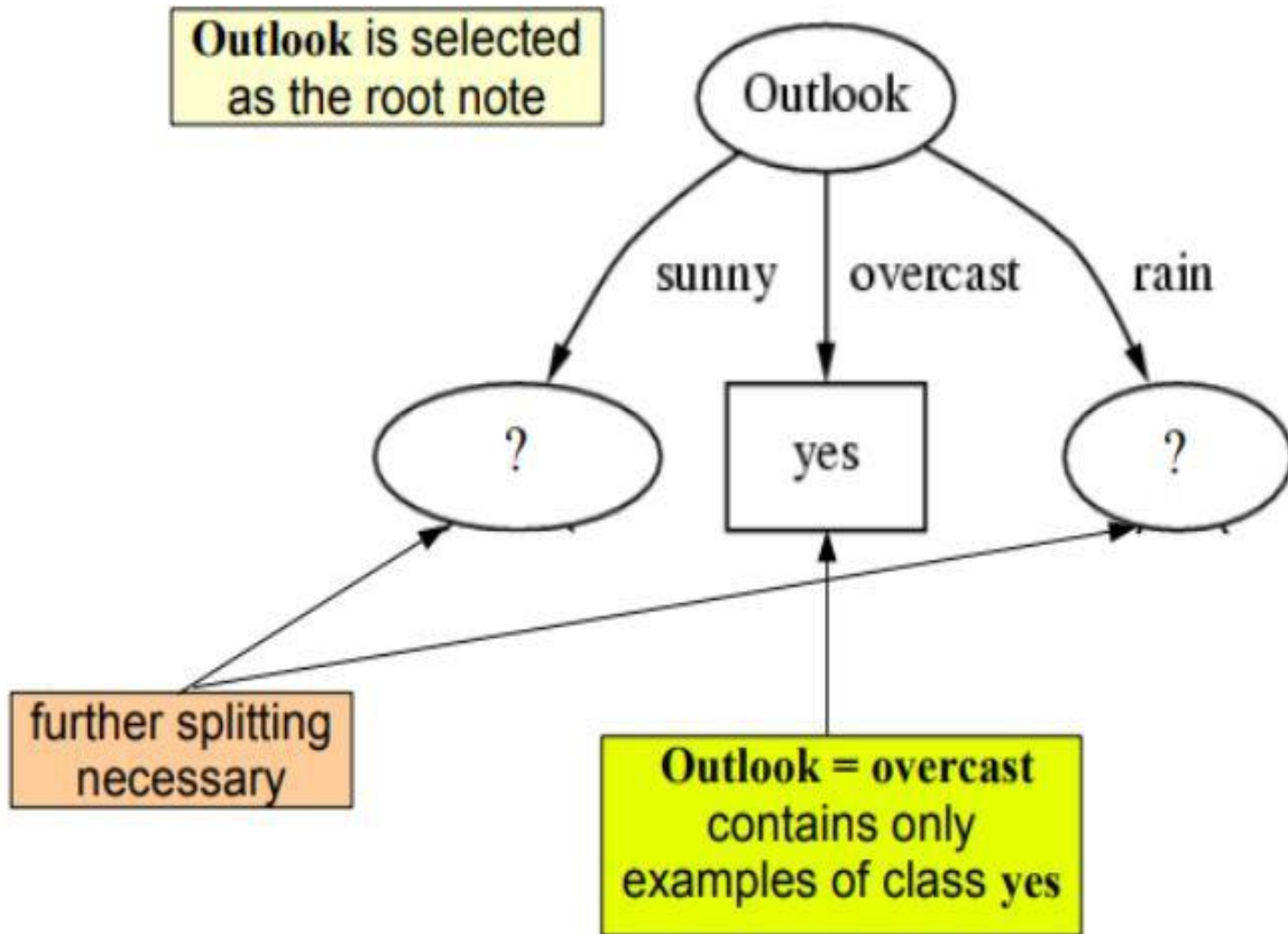
Entropy and Information Gain

- Similarity we can calculate for other two attributes (Humidity and Temp).

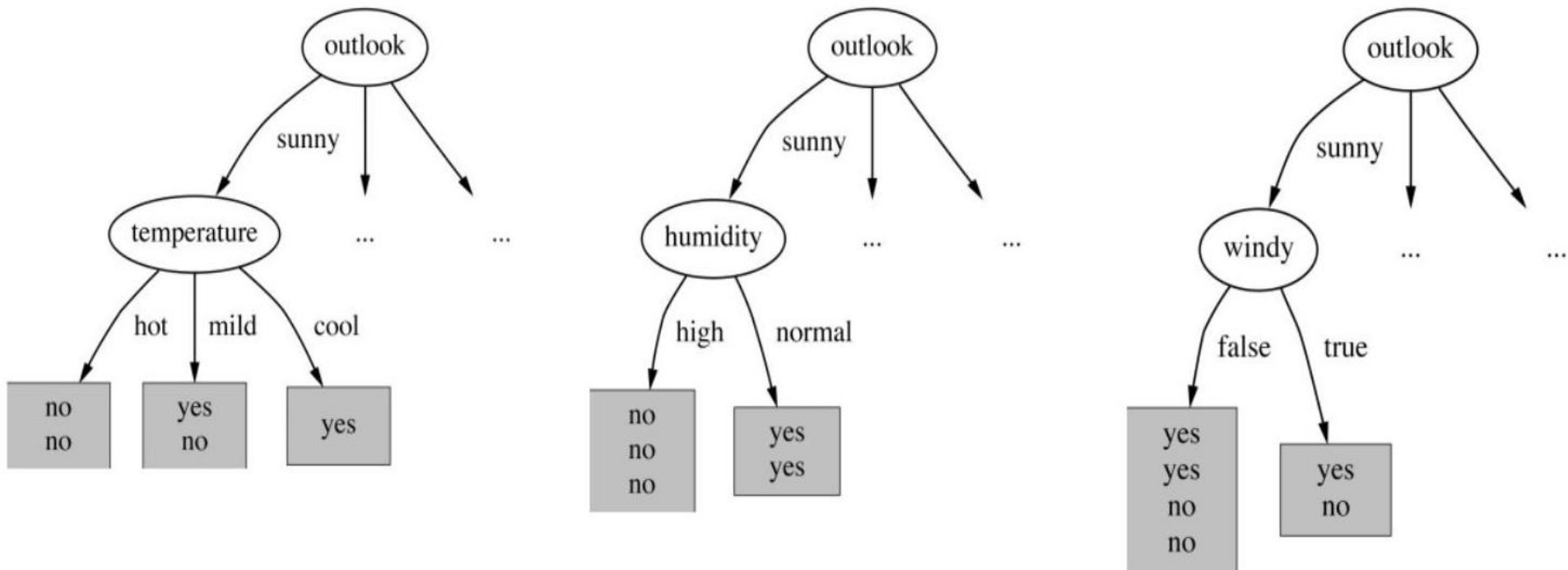
Pick the highest gain attribute

Outlook	Temperature
Info: 0.693 Gain: $0.940 - 0.693$ 0.247	Info: 0.911 Gain: $0.940 - 0.911$ 0.029
Humidity	Windy
Info: 0.788 Gain: $0.940 - 0.788$ 0.152	Info: 0.892 Gain: $0.940 - 0.892$ 0.048

Root Node



Repeat the same thing for sub-trees till we get the tree.

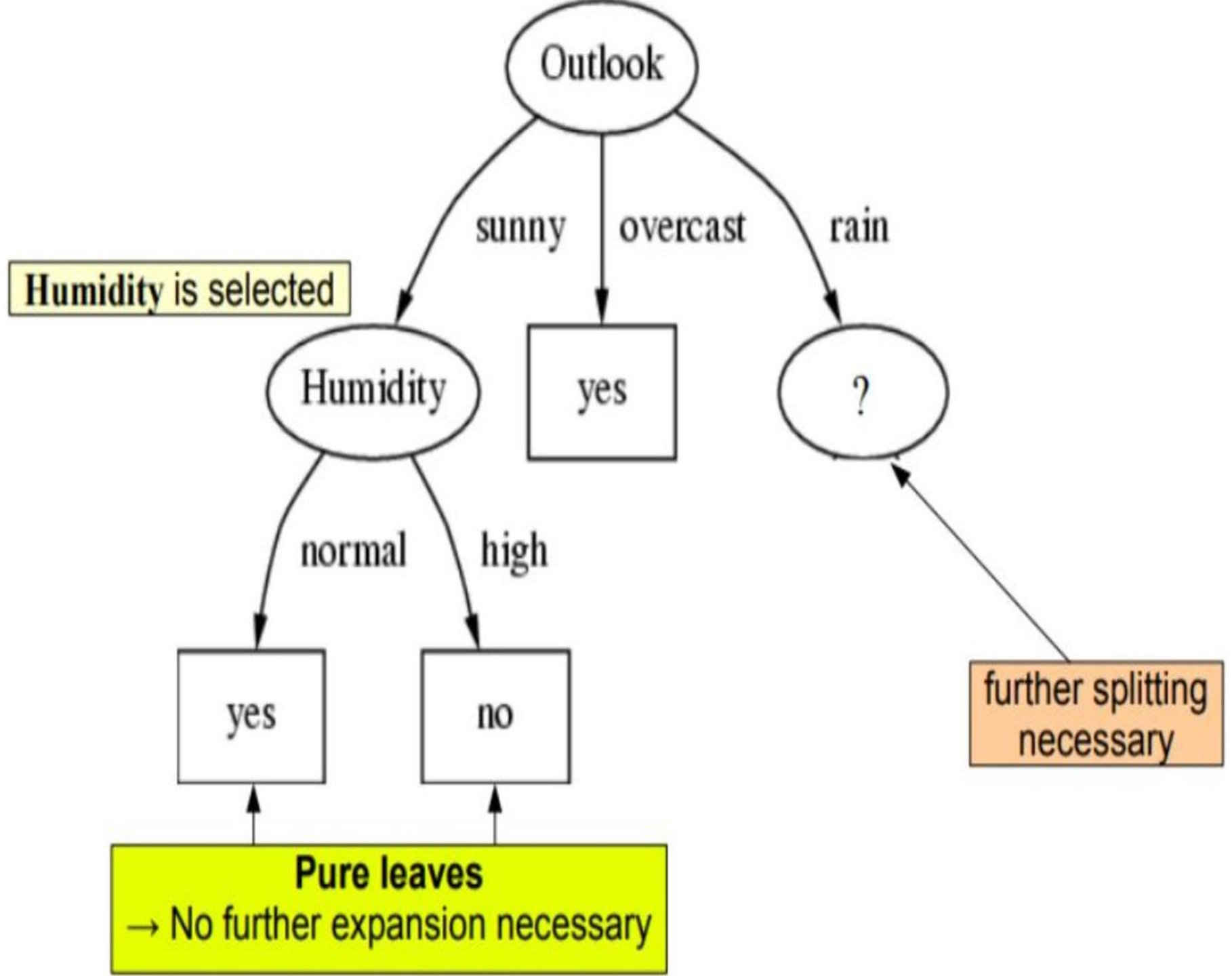


$\text{Gain}(\text{Temperature}) = 0.571 \text{ bits}$

$\text{Gain}(\text{Humidity}) = 0.971 \text{ bits}$

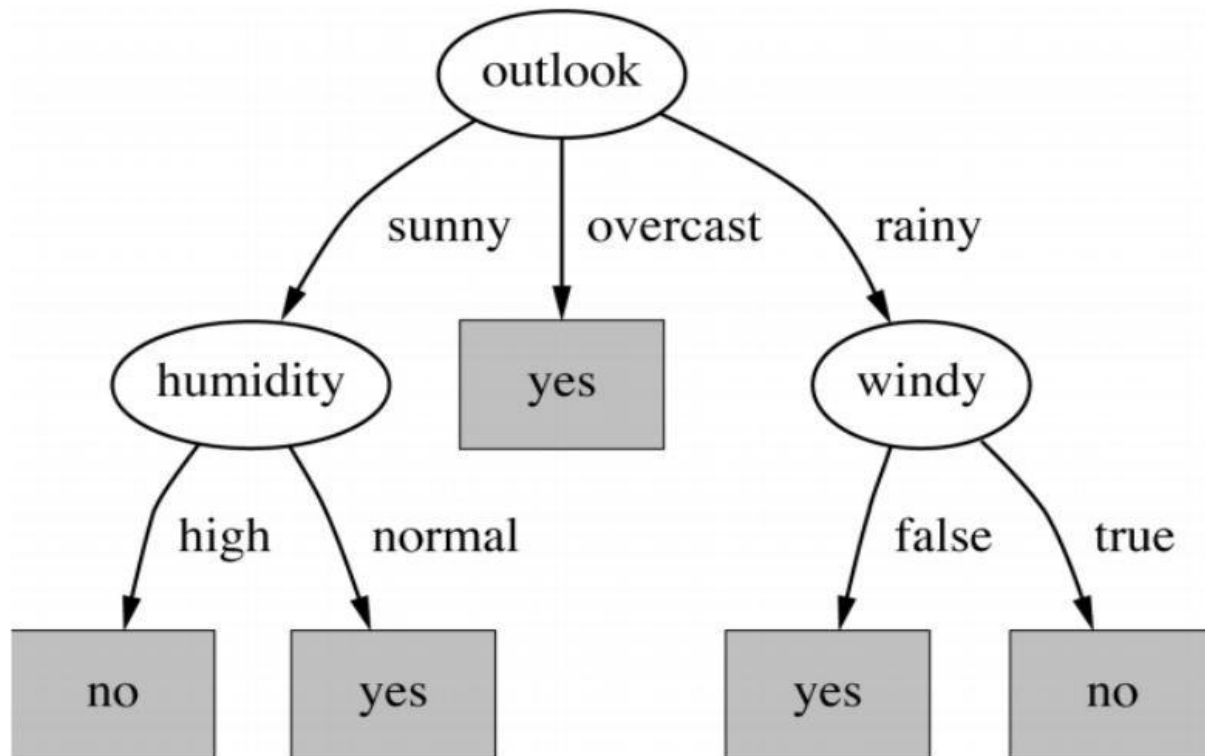
$\text{Gain}(\text{Windy}) = 0.020 \text{ bits}$

Humidity is selected



Final Decision Tree

Final decision tree



Overfitting

Overfitting is a practical problem while building a decision tree model. The model is having an issue of overfitting is considered when the algorithm continues to go deeper and deeper in the to reduce the training set error but results with an increased test set error i.e, Accuracy of prediction for our model goes down. It generally happens when it builds many branches due to outliers and irregularities in data.

- Two approaches which we can use to avoid overfitting are:
 - Pre-Pruning
 - Post-Pruning

Pre-Pruning

- In pre-pruning, it stops the tree construction bit early. It is preferred not to split a node if its goodness measure is below a threshold value. But it's difficult to choose an appropriate stopping point.

Post-Pruning

- In post-pruning first, it goes deeper and deeper in the tree to build a complete tree. If the tree shows the overfitting problem then pruning is done as a post-pruning step. We use a cross-validation data to check the effect of our pruning. Using cross-validation data, it tests whether expanding a node will make an improvement or not.
- If it shows an improvement, then we can continue by expanding that node. But if it shows a reduction in accuracy then it should not be expanded i.e, the node should be converted to a leaf node.

Decision Tree Algorithm Advantages and Disadvantages

Advantages

- Decision Trees are easy to explain. It results in a set of rules.
- It follows the same approach as humans generally follow while making decisions.
- Interpretation of a complex Decision Tree model can be simplified by its visualizations. Even a naive person can understand logic.
- The Number of hyper-parameters to be tuned is almost null.

Disadvantages

- There is a high probability of overfitting in Decision Tree.
- Generally, it gives low prediction accuracy for a dataset as compared to other machine learning algorithms.
- Information gain in a decision tree with categorical variables gives a biased response for attributes with greater no. of categories.
- Calculations can become complex when there are many class labels.